



Cyberscope

A *TAC Security* Company

Audit Report

Hero Arena Play

May 2026

Repository https://github.com/HaGameFi/HeroArena_contracts

Commit hash 4827a7215d6876ec73d84ab69ce732810595ffcb

Audited by © cyberscope

Table of Contents

Table of Contents	1
Risk Classification	3
Review	4
Audit Updates	4
Source Files	4
Overview	5
HeroArenaAvatars.sol	5
HeroArenaProfile.sol	5
HeroArenaMiningFactoryV1.sol	5
Findings Breakdown	6
Diagnostics	7
PNL - Profile NFT Lockup	8
Description	8
Recommendation	9
AAI - Ambiguous Avatar ID	10
Description	10
Recommendation	11
BT - Burns Tokens	12
Description	12
Recommendation	12
ENC - External NFT Custody	13
Description	13
Recommendation	14
IDI - Immutable Declaration Improvement	15
Description	15
Recommendation	15
MT - Mints Tokens	16
Description	16
Recommendation	16
MC - Missing Check	17
Description	17
Recommendation	17
MEE - Missing Events Emission	18
Description	18
Recommendation	18
ORD - Ownership Role Desync	19
Description	19
Recommendation	20
POP - Pending Owner Persists	21

Description	21
Recommendation	22
PSR - Price Slippage Risk	23
Description	23
Recommendation	24
L16 - Validate Variable Setters	25
Description	25
Recommendation	25
Functions Analysis	26
Summary	29
Disclaimer	30
About Cyberscope	31

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Audit Updates

Initial Audit

18 May 2026

Source Files

Filename	SHA256
HeroArenaProfile.sol	baeff2a1a5922e5681681e128a5ab5c19a1 9101fba5df6cd7363e83fb8af1361
HeroArenaMiningFactoryV1.sol	0f07122bacaf2d5c244678983d3c27f96b2 d82f4c9bdec861b1dd34e180e6dfe
HeroArenaAvatars.sol	69c58e8537a0db3e40bf05098af0da9a1ad e3ce18418d7ca6e49912ad70ea64f

Overview

The three contracts implement Hero Arena's profile and avatar layer. `HeroArenaAvatars` defines the enumerable ERC721 avatar collection, `HeroArenaMiningFactoryV1` deploys and sells those avatar NFTs for HAP, and `HeroArenaProfile` lets users register profiles, join teams, attach avatar or frame NFTs, and accumulate user or team points. Together, they support identity, cosmetic ownership, team membership, and point-based progression for the broader Hero Arena ecosystem.

HeroArenaAvatars.sol

`HeroArenaAvatars` is an enumerable ERC721 collection for Hero Arena avatar NFTs. The owner can mint NFTs to users, assign each token an `avatarId`, burn NFTs, and set metadata labels and creation timestamps for each avatar type. The contract tracks minted and burned counts per avatar ID, exposes batch lookups for avatar IDs and avatar metadata, and allows users or external consumers to enumerate all avatar token IDs owned by an address.

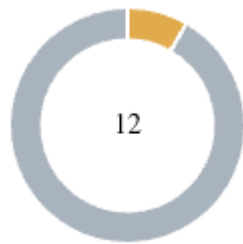
HeroArenaProfile.sol

`HeroArenaProfile` manages user profiles that are paid for in HAP. Users can register once, choose a joinable team, and later attach approved avatar or frame ERC721 tokens to their profile by transferring those NFTs into the contract. The owner manages teams, registration and update fees, and approved avatar or frame collections, while role-gated accounts can change user teams or adjust user and team points. The contract therefore acts as the identity, team, cosmetic, and points registry for Hero Arena users.

HeroArenaMiningFactoryV1.sol

`HeroArenaMiningFactoryV1` is the minting factory for Hero Arena avatar NFTs. At deployment it creates a new `HeroArenaAvatars` contract, initializes a fixed list of avatar names, stores the HAP mint price, and controls whether public claiming is available. Users can mint an avatar by paying the configured HAP price, while the owner can update claim availability, change the NFT price, withdraw collected HAP, and transfer ownership of the underlying avatar contract through a two-step pending-owner flow.

Findings Breakdown



● Critical	0
● Medium	1
● Minor / Informative	11

Severity	Unresolved	Acknowledged	Resolved	Other
● Critical	0	0	0	0
● Medium	1	0	0	0
● Minor / Informative	11	0	0	0

Diagnostics

● Critical ● Medium ● Minor / Informative

Severity	Code	Description	Status
●	PNL	Profile NFT Lockup	Unresolved
●	AAI	Ambiguous Avatar ID	Unresolved
●	BT	Burns Tokens	Unresolved
●	ENC	External NFT Custody	Unresolved
●	IDI	Immutable Declaration Improvement	Unresolved
●	MT	Mints Tokens	Unresolved
●	MC	Missing Check	Unresolved
●	MEE	Missing Events Emission	Unresolved
●	ORD	Ownership Role Desync	Unresolved
●	POP	Pending Owner Persists	Unresolved
●	PSR	Price Slippage Risk	Unresolved
●	L16	Validate Variable Setters	Unresolved

PNL - Profile NFT Lockup

Criticality	Medium
Location	contracts/HeroArenaProfile.sol#L220,L249
Status	Unresolved

Description

`updateAvatar()` and `updateFrame()` transfer the selected avatar or frame NFT from the user into the `HeroArenaProfile` contract. The previous NFT is returned only when the user replaces it with a new one. There is no separate function to remove the current avatar or frame and withdraw it without setting a replacement.

As a result, users cannot independently reclaim their currently assigned profile NFT. If a user no longer wants an avatar or frame attached, or if the approved NFT collection becomes unsupported, the NFT remains held by the contract unless the user has another valid replacement NFT and pays the update fee.

```
function updateAvatar(address _avatarAddress, uint256 _tokenId) external
{
    ...
    _userMapping[msg.sender].avatarAddress = _avatarAddress;
    _userMapping[msg.sender].tokenId = _tokenId;

    _avatarToken.safeTransferFrom(msg.sender, address(this), _tokenId);
    HapToken.safeTransferFrom(msg.sender, address(this), feeToUpdate);

    if (_previousAvatarAddress != address(0)) {
        IERC721(_previousAvatarAddress).safeTransferFrom(address(this),
msg.sender, _previousTokenId);
    }
}

function updateFrame(address _frameAddress, uint256 _frameTokenId)
external {
    ...
    _userMapping[msg.sender].frameAddress = _frameAddress;
    _userMapping[msg.sender].frameTokenId = _frameTokenId;

    _frameToken.safeTransferFrom(msg.sender, address(this),
_frameTokenId);
    HapToken.safeTransferFrom(msg.sender, address(this), feeToUpdate);

    if (_previousFrameAddress != address(0)) {
        IERC721(_previousFrameAddress).safeTransferFrom(address(this),
msg.sender, _previousFrameTokenId);
    }
}
```

Recommendation

Provide a dedicated removal or withdrawal flow for the currently assigned avatar and frame NFTs. Users should be able to detach and recover their profile NFTs without needing to supply a replacement NFT or depend on continued support for a specific collection.

AAI - Ambiguous Avatar ID

Criticality	Minor / Informative
Location	contracts/HeroArenaAvatars.sol#L18,L41,L72
Status	Unresolved

Description

`getAvatarIdBatch()` returns `_avatarIds[tokenId]` directly without checking whether each token exists. For non-existent or burned token IDs, the mapping returns the default value `0`. Since `avatarId 0` is a valid avatar type, callers cannot distinguish a real token assigned to avatar `0` from an invalid token ID.

This can cause frontends, profile systems, or external integrations to treat non-existent NFTs as valid avatar `0` NFTs if they rely on this batch lookup for validation or display.

```
mapping(uint256 => uint8) private _avatarIds;

function mint(address _to, uint8 _avatarId) external onlyOwner returns
(uint256) {
    _tokenCounter += 1;
    uint256 _newTokenId = _tokenCounter;
    _avatarIds[_newTokenId] = _avatarId;
    avatarCount[_avatarId] += 1;
    _mint(_to, _newTokenId);
    return _newTokenId;
}

function getAvatarIdBatch(uint256[] calldata _tokenIds) external view
returns (uint8[] memory) {
    uint8[] memory _Ids = new uint8[](_tokenIds.length);
    for (uint256 i = 0; i < _tokenIds.length; i++) {
        _Ids[i] = _avatarIds[_tokenIds[i]];
    }
    return _Ids;
}
```

Recommendation

Validate token existence before returning avatar IDs, or reserve `avatarId` `0` as an invalid sentinel value. Batch lookup functions should make invalid or burned token IDs explicit so consumers cannot confuse missing tokens with valid avatar metadata.

BT - Burns Tokens

Criticality	Minor / Informative
Location	contracts/HeroArenaAvatars.sol#L61
Status	Unresolved

Description

The contract owner has the authority to burn tokens from a specific address. The owner may take advantage of it by calling the `burn` function. As a result, the targeted address will lose the corresponding tokens.

```
function burn(uint256 _tokenId) external onlyOwner {
    uint8 _avatarId = _avatarIds[_tokenId];
    avatarCount[_avatarId] -= 1;
    avatarBurnCount[_avatarId] += 1;
    delete _avatarIds[_tokenId];
    _burn(_tokenId);
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

ENC - External NFT Custody

Criticality	Minor / Informative
Location	contracts/HeroArenaProfile.sol#L220,L249
Status	Unresolved

Description

`updateAvatar()` and `updateFrame()` accept NFT contract addresses supplied by the user, as long as those addresses have been granted `AVATAR_ROLE` or `FRAME_ROLE`. The contract then calls `ownerOf()` and `safeTransferFrom()` on the supplied NFT contract and stores the NFT in `HeroArenaProfile`.

Because the NFT contract controls the behavior of these external calls, unsafe or non-standard whitelisted NFTs can cause profile updates to revert, behave unexpectedly, or create custody and compatibility issues. The whitelist is therefore security-critical and should only include NFT contracts whose transfer behavior and ERC721 compliance are trusted.

```
function updateAvatar(address _avatarAddress, uint256 _tokenId) external
{
    require(hasRegistered[msg.sender], "User not registered");
    require(hasRole(AVATAR_ROLE, _avatarAddress), "Avatar address
invalid");

    IERC721 _avatarToken = IERC721(_avatarAddress);
    require(msg.sender == _avatarToken.ownerOf(_tokenId), "Only owner can
transfer his/her NFT");

    _userMapping[msg.sender].avatarAddress = _avatarAddress;
    _userMapping[msg.sender].tokenId = _tokenId;

    _avatarToken.safeTransferFrom(msg.sender, address(this), _tokenId);
    ...
}

function addAvatarAddress(address _avatarAddress) external onlyOwner {
    require(IERC721(_avatarAddress).supportsInterface(0x80ac58cd), "Not
ERC721");
    _grantRole(AVATAR_ROLE, _avatarAddress);
}
```

Recommendation

Treat avatar and frame whitelisting as a trusted integration process. Only approve NFT contracts that have been reviewed for ERC721 compliance, predictable transfer behavior, and safe interaction with profile custody. Consider documenting whitelist criteria and adding operational checks before granting avatar or frame roles.

IDI - Immutable Declaration Improvement

Criticality	Minor / Informative
Location	contracts/HeroArenaMiningFactoryV1.sol#L129
Status	Unresolved

Description

The contract declares state variables that their value is initialized once in the constructor and are not modified afterwards. The `immutable` is a special declaration for this kind of state variables that saves gas when it is defined.

```
numberOfAvatars
```

Recommendation

By declaring a variable as immutable, the Solidity compiler is able to make certain optimizations. This can reduce the amount of storage and computation required by the contract, and make it more gas-efficient.

MT - Mints Tokens

Criticality	Minor / Informative
Location	contracts/HeroArenaAvatars.sol#L41
Status	Unresolved

Description

The contract owner has the authority to mint tokens. The owner may take advantage of it by calling the `mint` function. As a result, the contract tokens will be highly inflated.

```
function mint(address _to, uint8 _avatarId) external onlyOwner returns
(uint256) {
    _tokenCounter += 1;
    uint256 _newTokenId = _tokenCounter;
    _avatarIds[_newTokenId] = _avatarId;
    avatarCount[_avatarId] += 1;
    _mint(_to, _newTokenId);
    return _newTokenId;
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

MC - Missing Check

Criticality	Minor / Informative
Location	contracts/HeroArenaMiningFactoryV1.sol#33
Status	Unresolved

Description

The contract is processing variables that have not been properly sanitized and checked that they form the proper shape. These variables may produce vulnerability issues.

```
constructor(IERC20 _HapToken, uint256 _price) Ownable(msg.sender) {  
    HapToken = _HapToken;  
    ...  
}
```

Recommendation

The team is advised to properly check the variables according to the required specifications.

MEE - Missing Events Emission

Criticality	Minor / Informative
Location	contracts/HeroArenaAvatars.sol#L53 contracts/HeroArenaProfile.sol#L147,L155,L163,L278,L286,L359,L374 contracts/HeroArenaMiningFactoryV1.sol#L192
Status	Unresolved

Description

The contract performs actions and state mutations from external methods that do not result in the emission of events. Emitting events for significant actions is important as it allows external parties, such as wallets or dApps, to track and monitor the activity on the contract. Without these events, it may be difficult for external parties to accurately determine the current state of the contract.

```
function setAvatarNameAndCreatedTimestamp(uint8 _avatarId, string
calldata _avatarName) external onlyOwner
function claimFee(uint256 _amount) external onlyOwner
function makeTeamJoinable(uint256 _teamId) external onlyOwner
function makeTeamNotJoinable(uint256 _teamId) external onlyOwner
function claimFee(uint256 _amount) external onlyOwner
function addAvatarAddress(address _avatarAddress) external onlyOwner
function addFrameAddress(address _frameAddress) external onlyOwner
function decreaseUserPoints(address _userAddress, uint256
_numberOfPoints) external onlyPoint
function decreaseUserPointsBatch(address[] calldata _userAddresses,
uint256 _numberOfPoints) external onlyPoint
function decreaseTeamPoints(uint256 _teamId, uint256 _numberOfPoints)
external onlyPoint
```

Recommendation

It is recommended to include events in the code that are triggered each time a significant action is taking place within the contract. These events should include relevant details such as the user's address and the nature of the action taken. By doing so, the contract will be more transparent and easily auditable by external parties. It will also help prevent potential issues or disputes that may arise in the future.

ORD - Ownership Role Desync

Criticality	Minor / Informative
Location	contracts/HeroArenaProfile.sol#L16,L21,L78
Status	Unresolved

Description

`HeroArenaProfile` inherits both `Ownable` and `AccessControl`. The constructor grants `DEFAULT_ADMIN_ROLE` to the initial owner, but there is no override of ownership transfer logic to keep the two authority systems synchronized. If ownership is transferred, the new owner receives `onlyOwner` permissions but does not automatically receive `DEFAULT_ADMIN_ROLE`.

This can leave the previous owner as the `AccessControl` admin, retaining the ability to grant or revoke roles such as `AVATAR_ROLE`, `FRAME_ROLE`, `POINT_ROLE`, and `SPECIAL_ROLE`. The new owner may control owner-only functions while being unable to administer roles, creating split governance and stale privileged access.

```
contract HeroArenaProfile is AccessControl, ERC721Holder, Ownable {
    bytes32 public constant AVATAR_ROLE = keccak256("AVATAR_ROLE");
    bytes32 public constant FRAME_ROLE = keccak256("FRAME_ROLE");
    bytes32 public constant POINT_ROLE = keccak256("POINT_ROLE");
    bytes32 public constant SPECIAL_ROLE = keccak256("SPECIAL_ROLE");

    constructor(IERC20 _HapToken, uint256 _feeToRegister, uint256
    _feeToUpdate) Ownable(msg.sender) {
        HapToken = _HapToken;
        feeToRegister = _feeToRegister;
        feeToUpdate = _feeToUpdate;

        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    }
}
```

Recommendation

Keep ownership and role administration aligned during ownership changes. Either use a single authority model consistently, or update ownership transfer handling so the new owner receives `DEFAULT_ADMIN_ROLE` and the previous owner's role is revoked when appropriate.

POP - Pending Owner Persists

Criticality	Minor / Informative
Location	contracts/HeroArenaMiningFactoryV1.sol#L23,L173
Status	Unresolved

Description

`proposeNFTContractOwnership()` stores a pending owner that can later call `acceptNFTContractOwnership()` to receive ownership of the `HeroArenaAvatars` contract. However, pending proposals are not invalidated when factory ownership changes. If the current owner proposes a pending NFT owner and then transfers ownership of the factory, the old pending address can still accept later and take ownership of the avatar contract.

This can surprise the new factory owner because control of the factory and control of the avatar NFT contract can diverge based on a stale proposal made before the ownership change.

```
address public pendingNFTContractOwner;

function proposeNFTContractOwnership(address _newOwner) external
onlyOwner {
    pendingNFTContractOwner = _newOwner;
    emit NFTContractOwnershipProposed(HeroArenaAvatarsSC.owner(),
    _newOwner);
}

function acceptNFTContractOwnership() external {
    require(msg.sender == pendingNFTContractOwner, "Not the pending
owner");
    address _previousOwner = HeroArenaAvatarsSC.owner();
    pendingNFTContractOwner = address(0);
    HeroArenaAvatarsSC.transferOwnership(msg.sender);
    emit NFTContractOwnershipTransferred(_previousOwner, msg.sender);
}
```

Recommendation

Invalidate pending avatar ownership proposals whenever factory ownership changes, or require the current factory owner to reconfirm pending transfers before they can be accepted. Pending ownership proposals should have clear lifecycle boundaries so stale approvals cannot outlive the authority that created them.

PSR - Price Slippage Risk

Criticality	Minor / Informative
Location	contracts/HeroArenaMiningFactoryV1.sol#L145,L161 contracts/HeroArenaProfile.sol#L170,L181,L220
Status	Unresolved

Description

`HeroArenaProfile.updateFeeCost()` and `HeroArenaMiningFactoryV1.updateNFTPrice()` allow the owner to change user-facing costs at any time. The affected user flows, including profile creation, avatar/frame updates, and NFT minting, always charge the current stored fee or price and do not let users specify the maximum amount they intended to pay.

A user may approve HAP and submit a transaction expecting one cost, but the owner can update the fee or NFT price before the transaction is mined. If the user granted sufficient allowance, the transaction can execute at the new higher amount, creating a fee-slippage and frontrunning risk.


```
function updateFeeCost(uint256 _feeToRegister, uint256 _feeToUpdate)
external onlyOwner {
    feeToRegister = _feeToRegister;
    feeToUpdate = _feeToUpdate;

    emit UpdateFeeCost(msg.sender, _feeToRegister, _feeToUpdate);
}

function createProfile(uint256 _teamId) external {
    ...
    HapToken.safeTransferFrom(msg.sender, address(this), feeToRegister);
}

function updateAvatar(address _avatarAddress, uint256 _tokenId) external
{
    ...
    HapToken.safeTransferFrom(msg.sender, address(this), feeToUpdate);
}

function updateNFTPrice(uint256 _newPrice) external onlyOwner {
    nftPrice = _newPrice;

    emit AvatarPriceUpdated(_newPrice);
}

function mintNFT(uint8 _avatarId) external {
    ...
    HapToken.safeTransferFrom(msg.sender, address(this), nftPrice);
}
```

Recommendation

Add user-side cost protection to all payable user flows, or introduce delayed administrative fee and price updates. Transactions should validate the user's expected maximum cost, and protocol-controlled price changes should provide enough notice before they become active.

L16 - Validate Variable Setters

Criticality	Minor / Informative
Location	contracts/HeroArenaMiningFactoryV1.sol#L174
Status	Unresolved

Description

The contract performs operations on variables that have been configured on user-supplied input. These variables are missing of proper check for the case where a value is zero. This can lead to problems when the contract is executed, as certain actions may not be properly handled when the value is zero.

```
pendingNFTContractOwner = _newOwner
```

Recommendation

By adding the proper check, the contract will not allow the variables to be configured with zero value. This will ensure that the contract can handle all possible input values and avoid unexpected behavior or errors. Hence, it can help to prevent the contract from being exploited or operating unexpectedly.

Functions Analysis

Contract	Type	Bases		
	Function Name	Visibility	Mutability	Modifiers
HeroArenaProfile	Implementation	AccessContr ol, ERC721Hold er, Ownable		
		Public	✓	Ownable
	addTeam	External	✓	onlyOwner
	getTeam	External		-
	renameTeam	External	✓	onlyOwner
	makeTeamJoinable	External	✓	onlyOwner
	makeTeamNotJoinable	External	✓	onlyOwner
	claimFee	External	✓	onlyOwner
	updateFeeCost	External	✓	onlyOwner
	createProfile	External	✓	-
	updateAvatar	External	✓	-
	updateFrame	External	✓	-
	addAvatarAddress	External	✓	onlyOwner
	addFrameAddress	External	✓	onlyOwner
	changeTeam	External	✓	onlySpecial
	increaseUserPoints	External	✓	onlyPoint
	increaseUserPointsBatch	External	✓	onlyPoint
	increaseTeamPoints	External	✓	onlyPoint

	decreaseUserPoints	External	✓	onlyPoint
	decreaseUserPointsBatch	External	✓	onlyPoint
	decreaseTeamPoints	External	✓	onlyPoint
	getUserProfile	External		-
HeroArenaMiningFactoryV1	Implementation	Ownable		
		Public	✓	Ownable
	updateAvailableClaim	External	✓	onlyOwner
	mintNFT	External	✓	-
	updateNFTPrice	External	✓	onlyOwner
	proposeNFTContractOwnership	External	✓	onlyOwner
	acceptNFTContractOwnership	External	✓	-
	claimFee	External	✓	onlyOwner
HeroArenaAvatars	Implementation	ERC721Enumerable, Ownable		
		Public	✓	ERC721 Ownable
	supportsInterface	Public		-
	_baseURI	Internal		
	mint	External	✓	onlyOwner
	setAvatarNameAndCreatedTimestamp	External	✓	onlyOwner
	burn	External	✓	onlyOwner
	getAvatarIdBatch	External		-

	getAvatarNameAndCreatedTimestamp Batch	External		-
	getTokensByOwner	External		-

Summary

Hero Arena contracts implement a profile and avatar suite built around user team registration, HAP-paid avatar NFT minting, cosmetic profile customization, and point-based progression. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

cyberscope.io